

Seleção Eficiente de Armas em Jogos Utilizando Lógica Fuzzy

Guilherme Oliveira Pimentel, Gabriel Daher Monteiro Bastos Nascimento, Pedro Henrique Kroll Nascimento, Daniel Ongarato de Arruda.

Resumo:

Este trabalho tem como objetivo mostrar como a lógica Fuzzy pode ser usada nos ambientes virtuais, por exemplo, em jogos, para tomar certas decisões. Num ambiente que envolve diversas variáveis e essas possuem diversos estados, há uma necessidade cada vez maior de especificar-se certas características para adequar-se melhor às inúmeras situações que os ambientes de jogos virtuais podem trazer. Com isso em mente, este grupo elaborou um projeto que mostra como a Lógica Difusa é essencial para auxiliar numa jogabilidade mais divertida e imersiva. Nesse artigo, mostraremos como a utilização de um modelo com diversos estados para diferentes variáveis constrói um ambiente virtual com mais versatilidade; serão apresentados modelos, funções, exemplos e, certamente, as motivações por trás dos elementos mostrados, sempre com o objetivo de dinamizar cada vez mais os jogos virtuais e melhorar a experiência dos usuários.

Palavras-chave:

1. INTRODUÇÃO

1.1. O Problema da Seleção de Armas em Jogos Digitais

Surgidos no final da década de 1950, os jogos eletrônicos têm ganhado cada vez mais espaço e notoriedade na sociedade. Por apresentar uma abordagem descontraída, envolvendo esquemas divertidos de jogatina, o

número de usuários tem aumentado constantemente com o passar dos anos. Esse cenário faz com que os *videogames*, como são popularmente conhecidos, sejam um dos principais meios de entretenimento na atualidade.

Por estar em constante evolução, alinhado com o avanço tecnológico, o desenvolvimento de novos jogos tem exigido cada vez mais recursos. Quanto maior a complexidade do *software*, mais os usuários tendem a se sentir confortáveis com a experiência de jogá-los. Isso pode ser percebido, principalmente, com a aplicação de gráficos potentes e mais realistas em comparação a jogos antigos, levando os jogadores a se sentirem mais inseridos no programa. Além disso, a jogabilidade exige constantes adaptações e melhorias em relação à quantidade de recursos alocados, evitando, assim, possíveis perdas de resolução, quedas de desempenho e falhas de programação. Assim, espera-se uma melhor experiência para o cliente, trazendo maior imersão ao jogar.

Nesse sentido, nota-se que diversos jogos da atualidade, notoriamente do estilo FPS (*First Person Shooter*), focado em combates com o uso de armas de fogo, apresentam problemas oriundos da pouca adaptabilidade e inovação. Em programas FPS, assim como na realidade, cada arma possui funcionalidade própria e diferentes contextos de aplicabilidade, a depender de fatores como distância, potência, ambiente, entre outros. Contudo, devido à baixa flexibilidade desses sistemas considerando o atual cenário de forte crescimento tecnológico, a seleção da melhor arma para um combate específico apresenta falhas, mostrando-se complexa e pouco intuitiva. Percebe-se, por exemplo, que os *non-playable characters* (NPC), por estarem mal programados, acabam por selecionar métodos de combate ineficientes, tornando a experiência de jogar mais irreal e frustrante para os clientes.

1.2. Lógica Fuzzy: Uma Abordagem para Decisões em Ambientes de Múltiplas Variáveis

No ambiente computacional, a lógica binária, na qual as variáveis com as quais estão sendo trabalhadas assumem apenas dois estados: 0 ou 1, também

chamados de “verdadeiro” e “falso” ou “alto” e “baixo”, assume papel protagonista nas estruturas internas dos computadores e, de modo geral, nos programas feitos nesses ambientes. É verdade, no entanto, que ela possui sérias limitações em representar verdades parciais ou valores intermediários, uma vez que trata somente dos campos extremos.

Nesse cenário, em 1965, o Dr. Lofti Zadeh da Universidade da Califórnia introduziu o termo de Lógica Fuzzy com sua proposta de teoria de conjuntos difusos. Diferente do cenário booleano, as variáveis podem assumir uma gama de valores entre 0 e 1, representando o grau de pertencimento a um conjunto: o número “1” indica pertencimento total e “0” não pertencimento total. Com essa abordagem, torna-se possível mapear estados indeterminados da realidade para o ambiente computacional, como a avaliação de condições subjetivas, por exemplo, a temperatura de uma sala (frio, morno, quente) ou o bem-estar de alguém (Muito bem, bem, normal, mal, muito mal), estados que não podem ser representadas através dos extremos binários 0 ou 1.

1.3. Objetivo e Motivação do Trabalho

O objetivo deste trabalho é investigar e tornar realidade a aplicação da lógica fuzzy no desenvolvimento de jogos, em especial, nos sistemas de seleção de armas para os non-playable characters (*NPC*), isto é, entidades autônomas que requerem um sistema de decisão próprio. Em jogos digitais, a escolha da arma ideal pode depender de múltiplas condições que se alteram dinamicamente, tais quais a distância até o alvo e o número de munições. Nesse sentido, a lógica fuzzy, por sua capacidade em lidar com um grande número de entradas, além de uma forma que se aproxima mais à humana de pensar em sistemas de controles, mostra-se uma solução eficiente para tratar o problema de forma flexível e adaptativa.

A realização deste trabalho foi inspirada em um exemplo apresentado de forma parcial no livro *Programming Game AI by Example*, que propôs uma solução particular utilizando C++. No entanto, este trabalho se propõe a expandir e adaptar essa abordagem para o desenvolvimento no ambiente da engine Unity, utilizando a

linguagem de programação C# (C Sharp), a fim de explorar novas possibilidades e implementações.

2. CONCEITOS CHAVES

2.1. Conjuntos Fuzzy

Os conjuntos fuzzy são generalizações dos conjuntos tradicionais da lógica booleana, mas enquanto nesse um elemento pode somente pertencer ou não pertencer, naquele os elementos possuem um grau de pertencimento ao conjunto, que varia entre 0 a 1. Esse conceito é especialmente útil em ocasiões nas quais os elementos estudados variam entre estados vagos e/ou subjetivos, como temperatura - “quente”, “frio”, “morno” - velocidade - “lenta”, “moderada”, “rápida” - ou felicidade - “alegre”, “triste” ou “neutro”.

Os valores dessas chamadas **variáveis linguísticas** podem ser definidas em sentenças de uma linguagem especificada construída a partir de diferentes componentes. Esses componentes incluem:

- Termos primários: descrevem as variáveis, como "grande", "pequeno", "lento", "rápido", "quente".
- Operadores lógicos: combinam termos linguísticos, como "e", "ou" e "negação".
- Modificadores: responsáveis por alterar a intensidade ou extensão de um termo linguístico, a exemplo de "extremamente", "levemente", "muito" e "pouco".
- Delimitadores: usados para estruturar a expressão ou modificar a forma como os conectivos lógicos operam.

A principal função delas é sistematizar a caracterização aproximada de fenômenos turbulentos ou complexos, os quais são difíceis de serem tratados pela matemática convencional.

2.2. Funções de Pertinência

A **função de pertinência** é a responsável por traduzir em primeira instância os valores das variáveis linguísticas em uma representação matemática, o que torna possível o tratar desses dados dentro de um sistema lógico bem estruturado.

Matematicamente, um conjunto fuzzy A pode ser definido da seguinte forma: dado um universo de discurso U, A de U é caracterizado por uma função de pertinência

$$\mu_A : U \rightarrow [0, 1]$$

Onde $\mu_A(x)$ indica o grau de pertencimento de um elemento $x \in U$ ao conjunto fuzzy A. Existem diferentes formas de funções de pertinência, escolhidas com base na natureza dos dados estudados e nas necessidades de modelagem. Algumas delas são:

Funções triangulares, fáceis de implementar e permitem a criação de fuzzy sets complexos. São definidas por uma linha ascendente, um pico e uma linha descendente. Podem ser geradas a partir de 3 valores reais. São dadas por:

$$\mu_A(x) = \begin{cases} 0, & x < a \text{ ou } x > c \\ \frac{x-a}{b-a}, & a \leq x < b \\ \frac{c-x}{c-b}, & b \leq x \leq c \end{cases}$$

Semelhantes às triangulares, as **funções trapezoidais** são fáceis de implementar e também permitem a criação de fuzzy sets complexos. Possuem uma região plano no topo, o que permite representar intervalos com pertinência máxima. Podem ser geradas a partir de 4 valores reais. São definidas como:

$$\mu_A(x) = \begin{cases} 0, & x < a \text{ ou } x > d \\ \frac{x-a}{b-a}, & a \leq x < b \\ 1, & b \leq x \leq c \\ \frac{d-x}{d-c}, & c \leq x \leq d \end{cases}$$

Funções gaussianas, representadas por uma curva suave, ideais para tratar de fenômenos contínuos. Elas são descritas por:

$$\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

Onde c é o ponto central e σ controla o afastamento da curva em relação ao centro.

Cada uma dessas funções gera gráficos com formatos distintos: v-shaped (em "V"), trapezoidal (em forma de trapézio) e bell-shaped (em formato de sino), respectivamente.

2.3 Regras de Decisão

Após o processo de fuzzificação, isto é, a passagem das condições reais para dentro de sistemas fuzzy e sua contextualização matemática, é possível utilizá-las em sistemas de tomadas de decisão. Um de seus principais pilares são as **regras de decisão**, responsáveis por determinar o comportamento do sistema ao relacionar as entradas com as saídas. Para isso, são compostas por condições lógicas, em especial as do tipo “SE (premissa é cumprida), ENTÃO (ação)”, nas quais as premissas definem as condições a serem avaliadas pelo sistema, e as ações o que deve ser feito com base nelas. Um exemplo simples num contexto qualquer é “se a velocidade é alta, então desacelerar”. A união das diferentes regras de decisão que compõem o sistema fuzzy pode ser chamada de **base de regras** e é fundamental para a etapa de **inferência fuzzy** dentro de um sistema de decisão.

2.4. Inferência Fuzzy

O método de inferência fuzzy interpreta o vetor gerado pelos inputs, utilizando a Base de Regras, e oferece uma saída. Isso é feito por meio da combinação de cada uma das regras usando operadores lógicos de conjunção e disjunção. Existem diferentes métodos de inferência que podem ser utilizados, no entanto, dois se destacam: o método de Mamdani, proposto em 1977, e o de Takagi-Sugeno, criado em 1985. Apesar de possuírem similaridades, a exemplo da fuzzificação das entradas e a aplicação do operador fuzzy, esses métodos apresentam diferenças que o tornam apropriados para diferentes tipos de aplicações.

O método de Mamdani é amplamente utilizado em sistemas de controle e é considerado um dos primeiros métodos de inferência fuzzy. Sua principal característica é que sua saída também é um conjunto fuzzy, o que permite um controle flexível. A saída efetiva do controlador é então obtida por meio de um processo de desfuzzificação aplicado no conjunto fuzzy. O método de Mamdani é amplamente utilizado em sistemas de controle simples e aplicações nas quais operadores humanos ajustam com frequência as regras, a exemplo deste projeto. Por outro lado, no método de Takagi-Sugeno, o resultado de cada regra é um valor numérico (não um conjunto fuzzy), que assume como peso o grau de pertinência obtido após avaliar as condições da regra fuzzy com base nos valores de entrada. Isso é especialmente útil para modelar sistemas não lineares e em operações que exigem alta precisão e eficiência computacional.

2.5 Defuzzificação

Após a saída do sistema de inferência fuzzy, o processo de desfuzzificação converte o resultado novamente para um valor preciso. Isso é essencial para que o resultado do processamento possa ser aplicado em contextos reais. Existem diferentes métodos de desfuzzificação, entre os quais o Método de Centro de Gravidade e a Média dos Máximos.

O Método de Centro de Gravidade (COG), também conhecido como Centróide, calcula o ponto de equilíbrio da área sob a curva de saída, o que retorna um valor exato que representa a média ponderada de todas as possibilidades. É amplamente utilizado devido à sua precisão em representar o valor fuzzy resultante, mas pode demandar maior capacidade computacional.

A Média dos Máximos (MOM), por sua vez, determina a saída segundo a média dos valores máximos de pertinência dentro do conjunto fuzzy. Embora esse método seja mais simples que o outro, é uma boa escolha para sistemas que demandam menor custo computacional.

A desfuzzificação é essencial para a aplicação dos resultados gerados pelo sistema de controle em suas etapas anteriores. A escolha do método mais adequado deve considerar o equilíbrio entre a precisão necessária, os recursos computacionais disponíveis, o que varia segundo o contexto no qual o sistema se insere.

3. METODOLOGIA

3.1. Definição das Variáveis Fuzzy

Com base no problema de decisão apresentado, escolhemos variáveis relevantes para o balanceamento recorrente de armas dentro de video-games, com o objetivo de fazer uma seleção dos equipamentos eficiente. Tais variáveis mais importantes são : a **distância entre a IA e seu alvo** (medida em metros) e a **quantidade de munição sobrando** para a arma a ser analisada.

Dessa forma, coletamos os dois valores necessários para cada arma dentro do arsenal de determinada IA e os fuzzificamos, colocando-os em vetores fuzzificados de distância e quantidade de munição.

Após isso, os valores estão prontos para o próximo passo, em que serão obtidos dados relacionados quanto à **desejabilidade** da arma.

3.2. Criação das Regras Fuzzy para Seleção de Armas

Para a criação das regras de inferência de cada uma das armas em posse da IA, é necessário fazer uma análise satisfatória em respeito ao funcionamento desejado para sua arma no jogo em questão. No projeto realizado para esse artigo, todas as armas apresentam cada uma 9 regras de inferência, as quais determinam a eficiência e desejabilidade da arma ser escolhida em determinada situação. O modelo utilizado de regras segue como : IF **X** AND **Y** THEN **Z**, onde as letras **X** e **Y** representam situações referentes às **variáveis fuzzy** das armas (Alvo_Longe, Alvo_Medio, Alvo_Perto, Munição_Pouca, Munição_Okay, Munição_Muito) e a letra **Z** significa a **desejabilidade** de tal arma (Indesejável, Desejável, Muito Desejável).

A forma mais correta de fazer regras precisas para as armas em um jogo é consultar jogadores experientes para analisar as regras oferecidas. Como exemplo, é esperado pelos jogadores recorrentes de *First Person Shooters* que uma espingarda não seja eficiente a longa distância devido a sua dispersão durante o disparo. Logo, não é indicado que escolha essa arma, a não ser que possua muita munição no inventário. De tal modo, as regras para uma arma como a Espingarda

certamente constariam parecidas com: IF **Alvo_Longe** AND **Munição_Pouca** THEN **Indesejável**, por exemplo.

3.3. Processo de Inferência e Defuzzificação

Após criar todas as regras de inferência das armas do arsenal da IA, é obtido dados fuzzificados de desejabilidade para cada uma das armas. Porém, os dados que são obtidos ainda não podem ser utilizados para fazer a seleção do equipamento, pois ainda estão em formato de vetor (fuzzificado). Para conseguir enfim comparar os dados de desejabilidade de cada arma, é necessário escolher um método de defuzzificação. Para esse projeto, o método selecionado foi o **Método de Centro de Gravidade (COG)**.

Finalmente, com a conclusão da função de defuzzificação, é obtido um valor float entre 0 e 100 que descreve a desejabilidade da arma, possibilitando a comparação entre a desejabilidade de outras armas presentes no arsenal e a seleção de arma de forma inteligente.

4 IMPLEMENTAÇÃO NO UNITY COM C#

4.1. Armas Escolhidas

Para a apresentação do projeto implementado na UNITY, as armas escolhidas para compor o arsenal da IA foram pensadas para cobrir a maior variedade de casos de equipamentos possíveis. Assim, foram escolhidas 3 armas com propriedades de distância e munição diversas. Tais armas são:

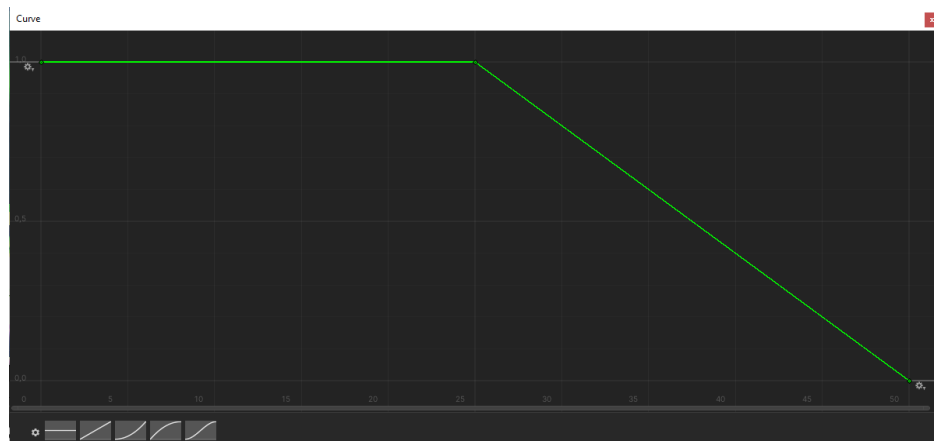
Pistola (ideal para média distância e bastante capacidade de munição);

Espingarda (ideal para curta distância e média capacidade de munição);

Fuzil de Precisão (ideal para longa distância e pouca capacidade de munição).

4.2. Gráficos Fuzzy Utilizando AnimationCurves da UNITY

Para utilizar os gráficos fuzzy na UNITY, uma ferramenta importante utilizada foi a AnimationCurve, um componente que pode descrever uma curva. Com isso, é possível juntar AnimationCurves para formar um gráfico no modelo padrão fuzzy.



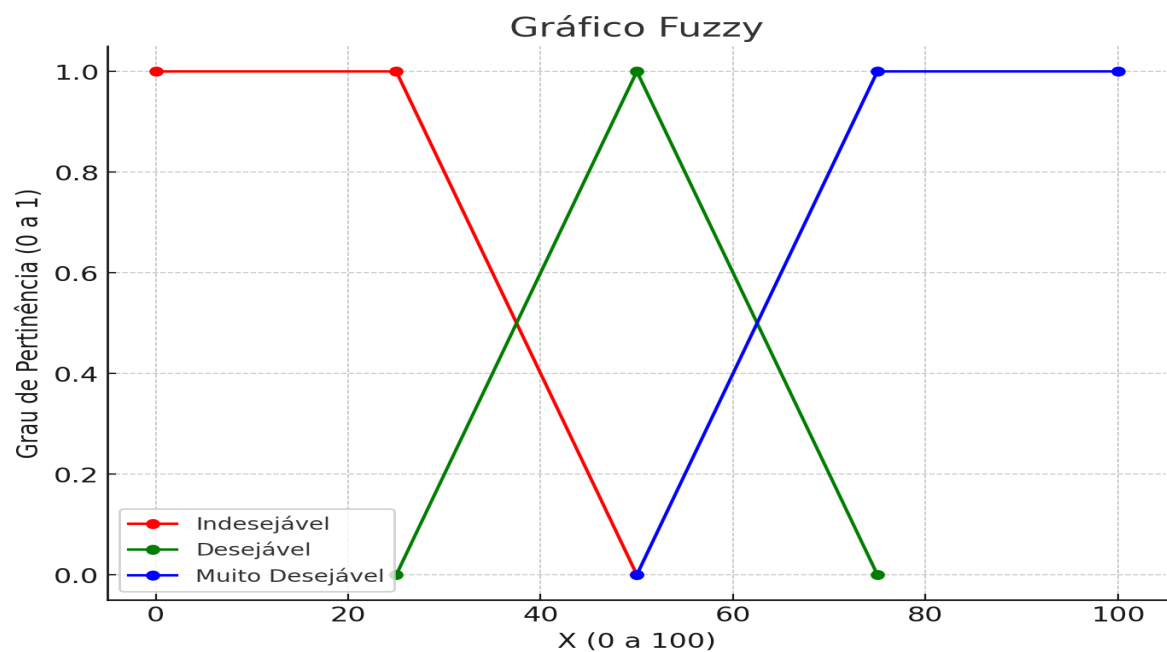
legenda: exemplo de AnimationCurve

A utilização de AnimationCurves é interessante durante a implementação com C#, pois os valores de X/Y podem ser facilmente encontrados a partir de funções da linguagem de programação dentro de scripts (ex: Evaluate).

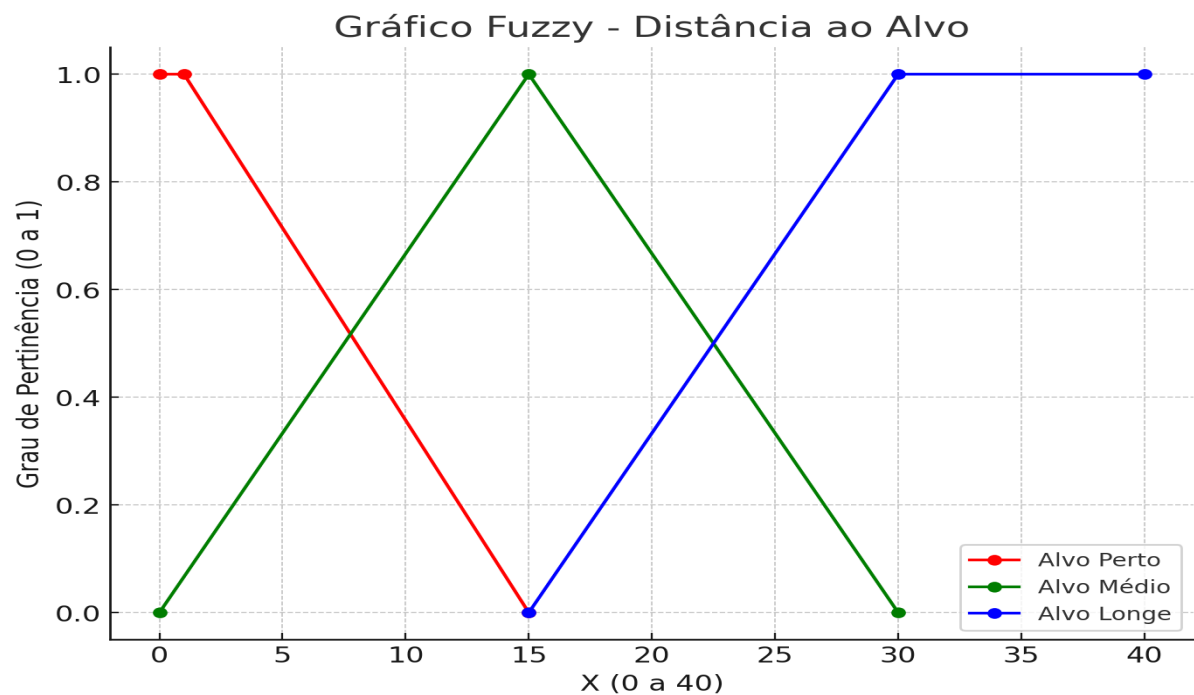
```
public override float[] FuzzifyAmmo() {  
    float[] fuzzifiedValues = new float[3];  
  
    fuzzifiedValues[0] = lowAmmoCurve.Evaluate(ammoCount);  
    fuzzifiedValues[1] = mediumAmmoCurve.Evaluate(ammoCount);  
    fuzzifiedValues[2] = highAmmoCurve.Evaluate(ammoCount);  
  
    return fuzzifiedValues;  
}
```

É válido ressaltar que os gráficos fuzzy que descrevem distância e desejabilidade são os mesmos para todas as armas. Isso ocorre pois, independentemente da arma que a IA esteja escolhendo, a distância entre ela e o alvo será a mesma entre todo o arsenal. Além disso, o gráfico de desejabilidade também será descrito de forma igual, visto que todas as armas precisam chegar em

um número de desejabilidade entre a mesma faixa de valores para ser possível a comparação de eficiência entre elas.



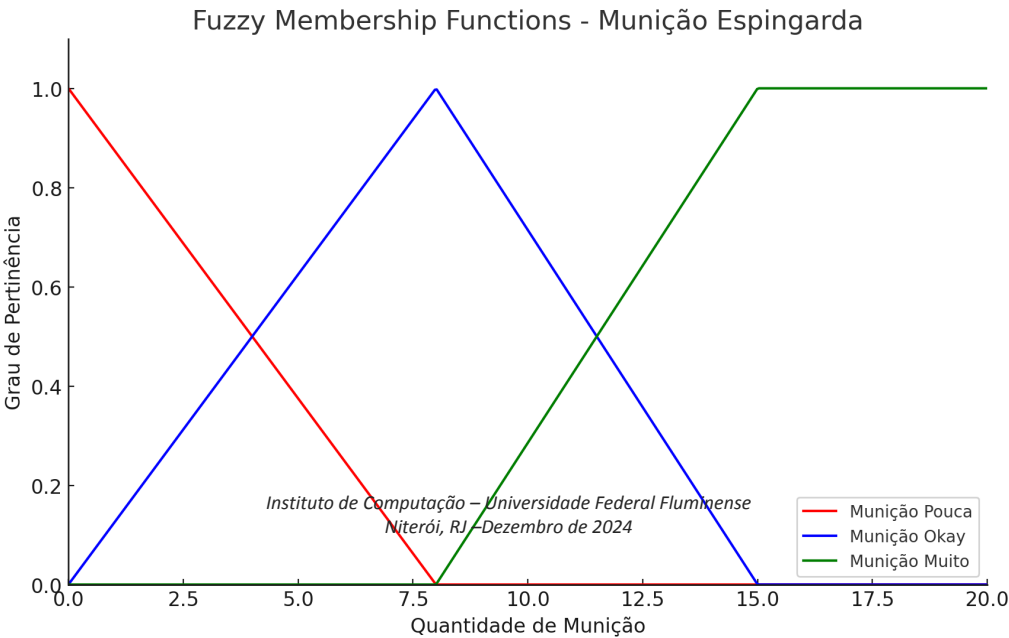
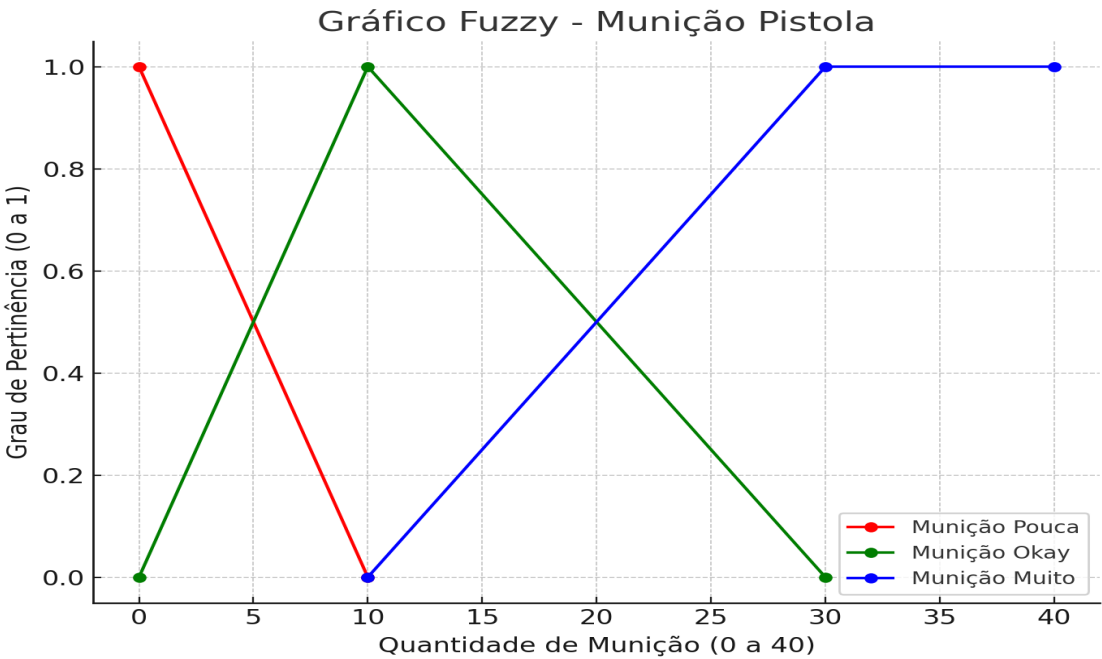
legenda: Gráfico Desejabilidade

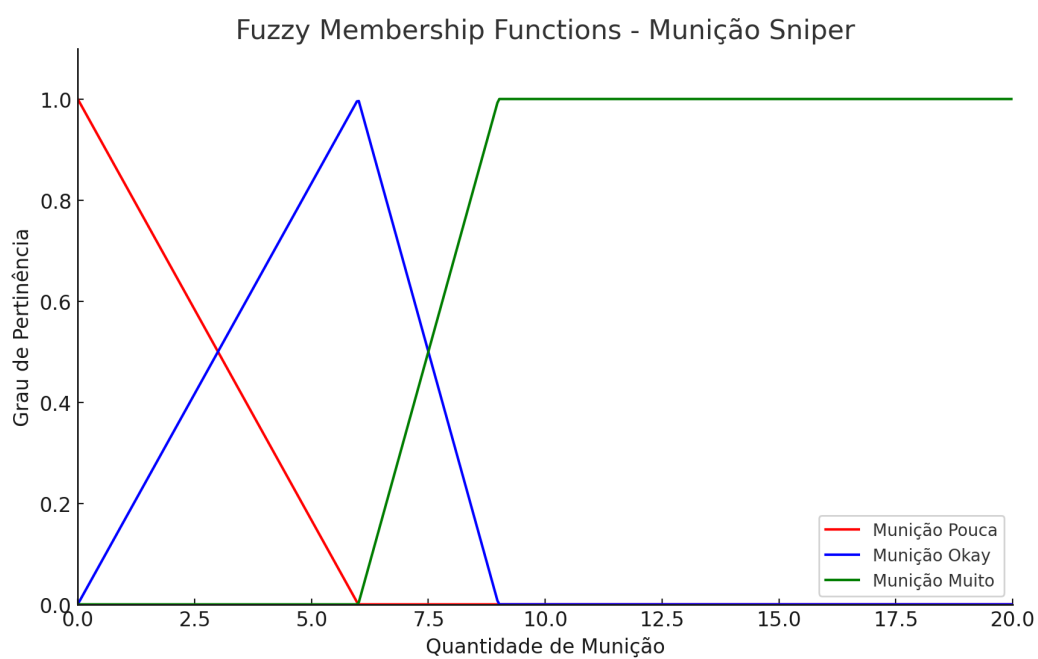


legenda: Gráfico Distância (em metros)

Assim como as regras de inferência, os gráficos fuzzy de distância e munição são construídos preferencialmente por uma base de jogadores experientes no funcionamento do jogo em questão.

4.2.1. Gráficos Fuzzy de Munição





4.3. Regras Fuzzy para Seleção de Armas

Como já anteriormente citado, cada arma necessita de algumas regras de inferência para produzir uma saída condizente com o resultado esperado. Nesse projeto, cada arma escolhida possui exatamente 9 regras.

Regras de Inferência da Pistola:

```
// Rule 1 IF Target_Far AND Ammo_Loads THEN Desirable
// Rule 2 IF Target_Far AND Ammo_Okay THEN Undesirable
// Rule 3 IF Target_Far AND Ammo_Low THEN Undesirable
// Rule 4 IF Target_Medium AND Ammo_Loads THEN VeryDesirable
// Rule 5 IF Target_Medium AND Ammo_Okay THEN VeryDesirable
// Rule 6 IF Target_Medium AND Ammo_Low THEN Desirable
// Rule 7 IF Target_Close AND Ammo_Loads THEN Undesirable
// Rule 8 IF Target_Close AND Ammo_Okay THEN Undesirable
// Rule 9 IF Target_Close AND Ammo_Low THEN Undesirable
```

Regras de Inferência da Espingarda:

```
// Rule 1 IF Target_Far AND Ammo_Loads THEN Undesirable
// Rule 2 IF Target_Far AND Ammo_Okay THEN Undesirable
// Rule 3 IF Target_Far AND Ammo_Low THEN Undesirable
// Rule 4 IF Target_Medium AND Ammo_Loads THEN Desirable
// Rule 5 IF Target_Medium AND Ammo_Okay THEN Undesirable
// Rule 6 IF Target_Medium AND Ammo_Low THEN Undesirable
// Rule 7 IF Target_Close AND Ammo_Loads THEN VeryDesirable
// Rule 8 IF Target_Close AND Ammo_Okay THEN VeryDesirable
// Rule 9 IF Target_Close AND Ammo_Low THEN VeryDesirable
```

Regras de Inferência do Fuzil de Precisão:

```
// Rule 1 IF Target_Far AND Ammo_Loads THEN VeryDesirable
// Rule 2 IF Target_Far AND Ammo_Okay THEN VeryDesirable
// Rule 3 IF Target_Far AND Ammo_Low THEN VeryDesirable
// Rule 4 IF Target_Medium AND Ammo_Loads THEN VeryDesirable
// Rule 5 IF Target_Medium AND Ammo_Okay THEN Desirable
// Rule 6 IF Target_Medium AND Ammo_Low THEN Undesirable
// Rule 7 IF Target_Close AND Ammo_Loads THEN Undesirable
// Rule 8 IF Target_Close AND Ammo_Okay THEN Undesirable
// Rule 9 IF Target_Close AND Ammo_Low THEN Undesirable
```

Ademais, há uma regra universal entre as armas para evitar a tentativa de uso de armas com a munição vazia. Essa regra faz com que o vetor fuzzificado de desejabilidade seja sobrescrito com um vetor que expressa 100% de indesejabilidade, independente da distância entre a IA e o alvo.

4.4. Processo de Inferência e Defuzzificação

Para o processo de inferência ser realizado em código C# e gerar o veto fuzzificado de desejabilidade, é necessário que os dois vetores fuzzificados de distância e munição já estejam disponíveis. Após começar a preencher o valor do vetor de desejabilidade, precisamos analisar os conectivos lógicos presentes nas regras de inferência. Para facilitar a compreensão, considere a regra 2 da pistola:

```
// Rule 2 IF Target_Far AND Ammo_Okay THEN Undesirable

desirabilityValues[Indesejável] =
Mathf.Max(desirabilityValues[Indesejável], Mathf.Min(distanceValues[Alvo_Longe,
ammoValues[Munição_Okay])));
```

Os valores de entrada (distância e munição) já foram fuzzificados em graus de pertinência.

`distanceValues[Alvo_Longe]` representa `Target_Far`.

`ammoValues[Munição_Okay]` representa `Ammo_Okay`.

O operador "**AND**" avalia o menor grau de pertinência entre as condições da regra. (implementado como `Mathf.Min`). O operador "**THEN**" ajusta o grau de pertinência do conjunto Undesirable, garantindo que ele represente o maior grau de ativação fornecido por qualquer regra. (implementado como `Mathf.Max`).

Em seguida, com a realização de todas as regras da arma, será obtido um vetor de desejabilidade que será defuzzificado pelo **Método de Centro de Gravidade (COG)**, enfim chegando em um valor possível de comparar com outras armas.

4.5. Seleção de Armas

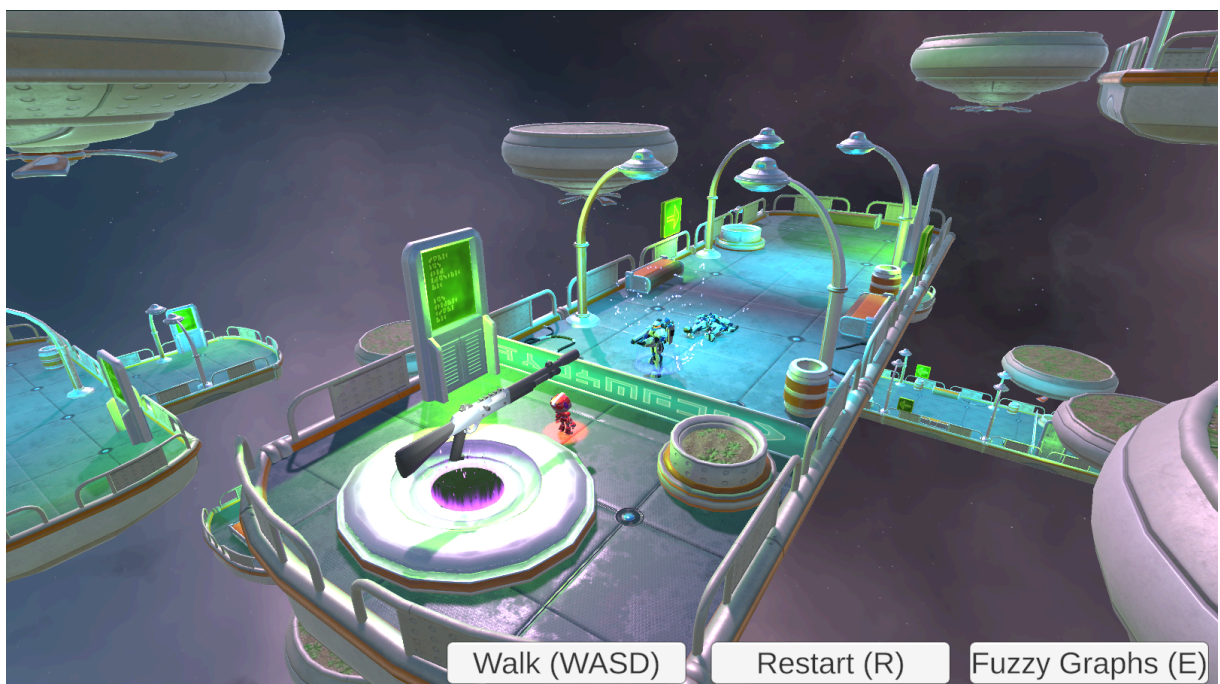
Para finalmente concluir o processo de seleção de armas inteligente para IA, basta comparar a desejabilidade entre as armas. Nesse caso, por exemplo, é possível utilizar um método como esse:

```
public void SelectWeaponByFuzzyLogic()
{
    if (sniperDesirability > shotgunDesirability && sniperDesirability >
pistolDesirability && _currentWeapon != sniper){
        SelectWeapon(sniper);
    }
    if (shotgunDesirability > sniperDesirability && shotgunDesirability >
pistolDesirability && _currentWeapon != shotgun){
        SelectWeapon(shotgun);
    }
    if (pistolDesirability > sniperDesirability && pistolDesirability >
shotgunDesirability && _currentWeapon != pistol){
        SelectWeapon(pistol);
    }
}
```

Nesse código C# é feita a verificação da necessidade da arma ser mudada levando em consideração a maior desejabilidade e a arma atualmente equipada (para evitar a tentativa de mudar para uma arma que já está equipada).

5. RESULTADOS PRÁTICOS

Para analisar os resultados obtidos a partir da aplicação da lógica fuzzy no processo de seleção de armas, foi produzido um cenário simples simulando uma IA em um campo de tiro dentro da UNITY(ver 2022.3.28f1).



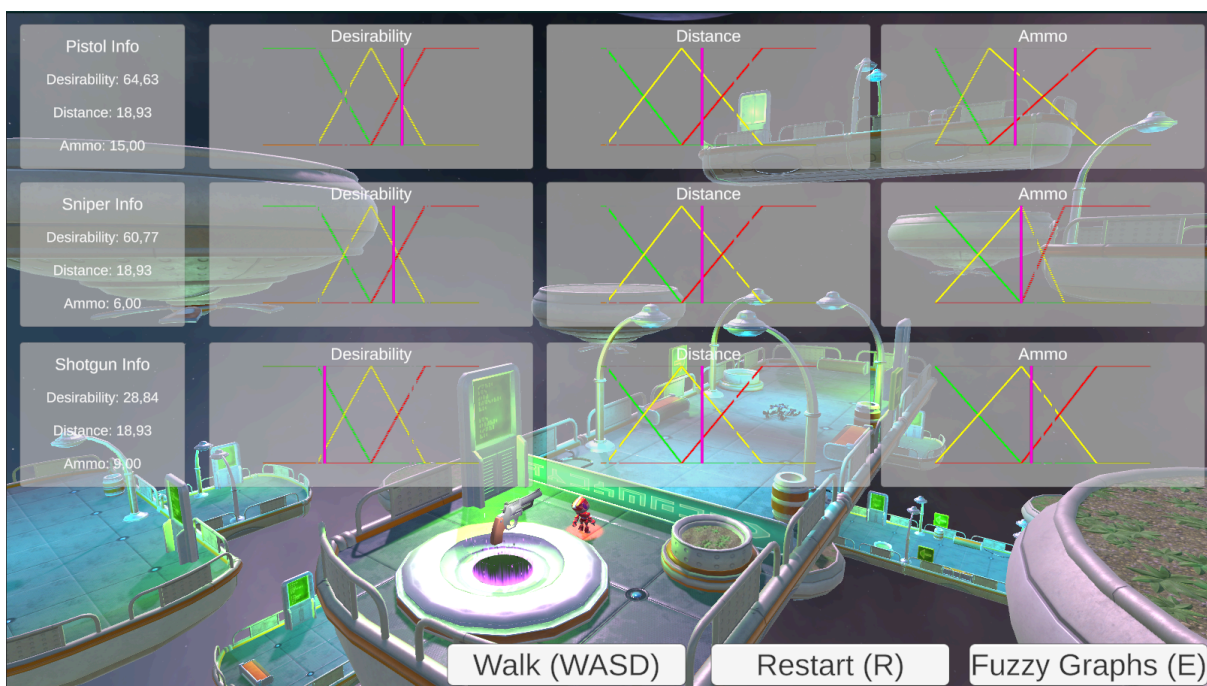
A IA (representada pelo personagem vermelho), faz as seleções inteligentes entre as 3 armas do seu arsenal com base na distância dos alvos e quantidade de munição restante em cada arma.

Os alvos (representados pelos robôs brancos) surgem aleatoriamente em uma distância entre 1 à 30 metros da IA. Esses alvos podem aparecer somente um de cada vez até que a munição da IA se esvazie completamente.

A munição de cada arma é gerada aleatoriamente entre um intervalo personalizado definido no código fonte. Após cada disparo, a munição e a desejabilidade são recalculadas e exibidas nos gráficos fuzzy.

Com os gráficos fuzzy, as análises ficam ainda mais precisas, habilitando a possibilidade de ver cada gráfico em tempo de execução. Além dos gráficos,

também é exibido simultaneamente a desejabilidade arma, distância exata do alvo atual e a quantidade de munição de todas as armas.



6. CONCLUSÃO

A aplicação da lógica fuzzy no processo de seleção de armas em jogos digitais, conforme apresentado neste trabalho, evidencia como essa abordagem pode melhorar significativamente a jogabilidade e a imersão. O uso de sistemas fuzzy permite que variáveis dinâmicas e subjetivas sejam tratadas de forma mais flexível e precisa, adaptando-se às complexidades do ambiente virtual. Por meio da definição de conjuntos fuzzy, funções de pertinência, regras de decisão e métodos de inferência, foi possível criar um modelo que simula escolhas mais humanas e realistas, superando as limitações dos sistemas tradicionais baseados na lógica binária.

Esse modelo representa um avanço importante na área de inteligência artificial aplicada a jogos, especialmente no controle de NPCs. Ao proporcionar decisões mais contextualizadas, como a seleção da arma ideal considerando fatores como distância e munição, a lógica fuzzy contribui para tornar as interações no jogo

mais envolventes e naturais. Além disso, a utilização da engine Unity e da linguagem C# possibilitou explorar a implementação prática do sistema, demonstrando sua viabilidade técnica e potencial para ser expandido a outras aplicações dentro do universo dos games.

Em resumo, a lógica fuzzy mostrou-se uma ferramenta poderosa para aprimorar a tomada de decisão em jogos digitais, equilibrando eficiência computacional com realismo. Este trabalho não apenas valida a aplicabilidade desse paradigma, como também abre caminhos para futuras pesquisas, incentivando a exploração de novos métodos e cenários onde a lógica fuzzy possa oferecer soluções inovadoras.

ASSETS UNITY GRATUITOS UTILIZADOS (ASSET STORE):

<https://assetstore.unity.com/packages/2d/textures-materials/sky/animated-fantasy-planet-98091>

<https://assetstore.unity.com/packages/vfx/particles/spells/magic-effects-free-247933>

<https://assetstore.unity.com/packages/3d/characters/jammo-character-mix-and-jam-158456>

<https://assetstore.unity.com/packages/3d/environments/sci-fi/sky-city-lite-65679>

<https://assetstore.unity.com/packages/3d/characters/robots/robot-hero-pbr-hp-polyart-106154>

<https://assetstore.unity.com/packages/vfx/particles/hit-impact-effects-free-218385>

<https://assetstore.unity.com/packages/3d/props/weapons/weapons-free-low-poly-asset-pack-by-ithappy-260492>

REFERÊNCIAS

LOFTI Zadeh, A. *Fuzzy inference: The process of formulating the mapping*. Disponível em: <https://www.sciencedirect.com/topics/engineering/fuzzy-inference>. Acesso em: 14 dez. 2024.

VALLE, M. E. *Introdução aos Sistemas Fuzzy*. Palestra realizada na UNESP Sorocaba, 2015. Disponível em: https://www.ime.unicamp.br/~valle/PDFfiles/Palestra_Sorocaba2015.pdf. Acesso em: 14 dez. 2024.

UNIVERSIDADE FEDERAL DE SANTA CATARINA. *Notas de Aula: Lógica Nebulosa e Sistemas Fuzzy*. Disponível em: <https://www.inf.ufsc.br/~mauro.roisenberg/ine5377/Cursos-ICA/LN-Sistemas%20Fuzzy.pdf>. Acesso em: 14 dez. 2024.

NASCIMENTO, P. H. K.; NASCIMENTO, G. D. M. B.; DE ARRUDA, D. O. *Seleção Eficiente de Armas em Jogos Utilizando Lógica Fuzzy*. Disponível em: <https://drive.google.com/file/d/1tpSTNm-5bu1hgtjmzntG5v-lsk2PSBvM/view>. Acesso em: 14 dez. 2024.